

Akanksha Madhu Kiran

Section: CMSC471(02-LEC) mon & wed 4:00PM

Question 1:

For each of the three cost measures—steps, Scrabble, and frequency—describe the heuristic implemented in your program.

For each heuristic:

1. Give a precise formula or pseudocode.
2. Explain what unavoidable remaining cost it lower-bounds.
3. Explain why it cannot overestimate the true optimal remaining cost.

Steps: $\text{step cost} = \text{number of positions where current word and the goal word disagree.}$

True cost is the number of moves as it costs 1 in each move and the number cannot be smaller than the number of wrong positions. Heuristic can only undercount or match the accurate cost. I can only change one letter at a time. So overestimating the cost is not possible.

Scrabble: add Scrabble values of the goal's letters in each mismatched position.

Final cost would be the cost of the goal letters that have been inserted at the position that is currently wrong. One move only gives a cost for that letter inserted at that place so costs are separate when multiple letters need to be changed. Scrabble values are all positive. So the real cost is always at least the sum. It cannot be overestimated since I am only adding the costs by counting how much it costs for the letters I need to change, it takes minimum number of fixing steps. If any changes need to be made it adds to the cost. Lower bounds the unavoidable remaining cost because task fixes position of mismatch must place the goal letter there, and that action's cost is exactly the value.

Frequency: The cost of a move is 1 plus the rarity of the word your goal is on. $\text{Cost} = 1 + \text{rarity}(\text{new word})$. The last move ends in the goal itself, so it costs exactly 1 plus the goal's rarity. Because other moves that are legal words of the same length cost as well. So it can only push the true cost higher, not lower. Every extra move would increase cost. Lower bounded by $1 + \text{minimum rarity}(\text{new word})$. The heuristic assumes the remaining moves are as cheap as possible. Therefore, the actual optimal cost can only be equal to or greater than the heuristic, never smaller. It cannot be overestimated as it takes minimum rarity.

Question 2:

Calculate the percentage reduction as:

$$100 \times (\text{UCS expansions} - \text{A}^* \text{ expansions}) / \text{UCS expansions}$$

A negative result means that A* expanded more nodes than UCS.

Then answer:

1. Did UCS and A* always return the same optimal cost? If not, identify and investigate the discrepancy.

Yes, UCS and A* always returned the same optimal cost

2. For each word pair, under which cost measure did A* produce the largest percentage reduction in expanded nodes?

Scrabble, is where A* produced the largest percentage reduction in expanded nodes for both pairs(why to ask, and bear to duck)

3. For each word pair, under which cost measure was the heuristic least informative?

Frequency is the least informative one for both why to ask and bear to duck pairs

4. Explain the differences using the definitions of your heuristics, not runtime alone.

Steps and Scrabbles measure the cost of mismatched letters directly so they give the best remaining cost. Scrabble is very effective as it gives the actual cost of goal letters.

Frequency heuristic is less informative as it considers minimum rarity(how rare the words are) for all words with the same length. It can be much smaller than the actual costs of the words, giving more A* expansions.

Word Pair	Cost Measure	UCS cost	UCS nodes expanded	A*cost	A*nodes expanded	Reduction
Why to Ask	Steps	5	773	5	16	97.93%
Why to Ask	Scrabble	9	756	9	13	98.28%
Why to Ask	Frequency	38.855	808	38.855	634	21.53%
Bear to Duck	Steps	4	691	4	6	99.13%
Bear to Duck	Scrabble	11	2952	11	6	99.80%
Bear to Duck	Frequency	38.240	1084	38.240	257	76.29%

Question 3

DOGCAT actions are reversible: if one legal word can be changed into another, the reverse change is also legal.

For each cost measure—steps, Scrabble, and frequency—determine whether the optimal cost from W_1 to W_2 must always equal the optimal cost from W_2 to W_1 .

For each cost measure:

1. State yes or no.
2. Justify your conclusion.
3. If the answer is no, give a concrete counterexample using words from the provided dictionary.

Steps: Yes, symmetric. Cost of an action is always 1 no matter what the direction is. Reversing would mean that a path just reverses the sequence, it would still be the same length, same total cost.

Scrabble: No, not symmetric. Cost of going from W1 to W2 depends on the end(final) letter. As path cost is the value of the newly inserted letter. The reverse path W2 to W1 charges the value of the letters being inserted going back. Counter example: W1 = just, W2 = must, to change from W1 to W2 it would cost the cost of m which is 3, for W2 to W1 it would cost the cost of j which is 6 and it is not equal.

Frequency: No, not symmetric, for the same reason. Cost of going from W1 to W2 depends on the end(final) letter. As path cost is the value of the newly inserted letter. The reverse path W2 to W1 charges the value of the letters being inserted going back. Counter example: W1 = just, W2 = must, to change from W1 to W2 it would cost the cost of m which is 3, for W2 to W1 it would cost the cost of j which is 6 and it is not equal.

Question 4

Question 4: LLM heuristic audit [10 points]

Give a generative AI system the following prompt:

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Submit:

1. The model used, the exact prompt, and its proposed heuristic. A complete conversation transcript is not required.
2. A precise formula or pseudocode for the proposed heuristic.
3. Determine whether the heuristic is admissible. Give either a general proof or a concrete counterexample.
4. If the proposal is invalid, make the smallest repair you can or replace it with a safe alternative.
5. Test the verified or repaired heuristic against h_0 on the two frequency-cost problems from Question 2. Compare solution costs and numbers of expanded nodes, then conclude whether the evidence supports using the proposed heuristic.

Experimental tests can reveal counterexamples, but passing a collection of tests does not by itself prove admissibility.

The AI system is not expected to be correct. Your grade is based on the quality of your verification, correction, and justification.

1. The model used is Gemini 3.6 Flash model. The exact prompt asked is the screenshot of the whole question is pasted below:

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Proposed heuristic is below:

$$h_1(n) = \begin{cases} 0 & \text{if } n = g \\ (h_0(n) - 1)(1 + r_{\min}) + (1 + \text{rarity}(g)) & \text{if } n \neq g \end{cases}$$

2. Precise formula or pseudocode for the proposed heuristic is:

```
def h_proposed(state, goal, dictionary):  
    # Calculate Hamming distance h_0  
    h0 = sum(1 for a, b in zip(state, goal) if a != b)  
  
    # Global minimum rarity across all valid words in the dictionary  
    min_rarity = min(word.rarity for word in dictionary)  
  
    # Admissible lower bound under step costs  
    return h0 * (1 + min_rarity)
```

3. Heuristic is admissible. Heuristic is:

$h(n) = c + (1 + \text{rarity_new})$ where $c = (h_0(n) - 1)(1 + r_{\min})$, where h_0 = number of mismatched positions from goal, $r_{\min}$ is the minimum rarity among the valid words, rarity_new is the rarity of the goal word. Any solution requires at least as much as minimum rarity. Heuristic can underestimate the cost or match it, but it cannot guess a cost that is too high.

4. In Gemini's heuristic precise formula pseudocode it missed the $(1 + \text{rarity of new word})$. It assumed the least possible rarity being the true one. I don't even think it gave the correct pseudocode for the proposed heuristic. In the pseudocode, if $r_{\min}$ is 0 then new and previous heuristic would be the same and it would not be informative. However the proposed heuristic is pretty good.
5. All heuristics return the same optimal cost as (confirming admissibility). I tested it by creating another run file to compare both heuristics to check if it gives the same solution.

```
[amadhuk1@linux1 program_assignment1]$ python hcheck.py
why -> ask
Heuristic: h0
Cost: 38.855
Time: 1.658
Solution: why way was has his ais ass ask

Heuristic: repaired frequency
Cost: 38.855
Time: 1.576
Solution: why way was has his ais ass ask

bear -> duck
Heuristic: h0
Cost: 38.24
Time: 2.735
Solution: bear beak beck buck duck

Heuristic: repaired frequency
Cost: 38.24
Time: 0.462
Solution: bear beak beck buck duck
```

Time is faster for repaired, however the cost is the same for both. Using repaired heuristic gives optimal solution in lesser time, but the normal old heuristic gives same solution but taking more time.